Fundamental Principles of Object-Oriented Design

Katerina Mangaroska week 37

01

GRASP Design Principles in OOP

General Responsibility Assignment Software Patterns (GRASP) is a set of software design principles that helps distribute responsibilities among objects in object-oriented systems.

The core idea is to allocate responsibilities between classes in a way that makes system easily maintainable, extensible, and modifiable, while also making the application code more understandable.

Applying these principles leads to reduced coupling between components and more efficient system architecture.

I. Information expert

Assign a responsibility to the class that has the necessary information to fulfill it, promoting high cohesion and minimizing coupling.

Responsibility of Information Expert is to encapsulate knowledge within itself and act as a single source of truth for its functionality.

For example: The library management system: when a user wants to borrow a book, the responsibility of checking if the book is available should lie with the Book class itself. The Book class contains information about its availability and can perform the necessary checks without needing to rely on other classes. This promotes high cohesion and reduces coupling between classes.

II. Creator principle

Assign the responsibility of creating instances of a class to the class that has the most knowledge about when and how to create them.

For example, the Customer class can create an Order object since a customer is logically responsible for placing orders.

```
class Customer {

    public Order placeOrder() {
        return new Order(this);
    }
}
```

```
class Order {
    private Customer customer;

    public Order(Customer customer) {
        this.customer = customer;
    }
}
```

II. Creator principle

The statement “Customer can create an Order object”

means that Customer may be the class that invokes the constructor because it has the GRASP Creator responsibility. It does not mean the constructor is written under Customer.

It means Customer is responsible for calling: new Order(...), while the constructor itself remains declared inside the Order class.

The created class owns its constructor; the Creator class decides when to call that constructor.

III. Controller

Assign the responsibility of handling system events or coordinating activities to a controller class, promoting centralized control and avoiding cluttered classes.

a distinct component should be responsible for orchestrating the actions of other components within a system. It should handle requests and events and delegate tasks to the appropriate objects.

For example: In a web application for a library, when a user requests to borrow a book, the responsibility of handling this request and coordinating the necessary actions should lie with a BorrowBookController class. This controller class would interact with other classes, such as Book, User, and Loan, to facilitate the borrowing process. By centralizing control logic in a controller class, the system becomes more organized and easier to manage.

III. Controller

Several software design patterns that leverage the Controller principle:

MVC: The Controller manages user input, updates the Model (data), and selects the appropriate View (presentation).

In Domain-Driven Design, the role of the Controller:

Application Coordination: Controller handles user input and maps it to domain operations, coordinates tasks, managing the sequence of interactions, ensuring that the domain logic remains decoupled from the UI and infrastructure logic.

Creator principle and Controller principle

If creating an order is simple and mainly depends on the customer, then Customer can reasonably be the GRASP Creator.

If placing an order requires coordinating many other things: inventory, payment, shipping, discounts, validation, then a controller is usually better.

orderController - new Order(customer, items, payment, promotion...)

GRASP Controller: who coordinates the whole use case?

GRASP Creator: who should invoke creation of the new object?

Constructor: always defined in the class being created.

IV. High Cohesion

Ensure that the responsibilities within a class are closely related and focused, enhancing readability, maintainability, and reusability.

…..the degree of consistency and connectivity between elements within a component. The more elements are logically related and affect each other, the higher the cohesion.

Example: In the library management system, the Book class should have cohesive responsibilities related to managing book details, such as title, author, and availability. Responsibilities unrelated to book management, such as user authentication, should be handled by separate classes. This ensures that each class is focused on a specific aspect of the system, promoting clarity and maintainability.

High Cohesion

- Low cohesion at Class level

Employee
SendEmail() GetSalary() PrintHolidays() SetPromotion()



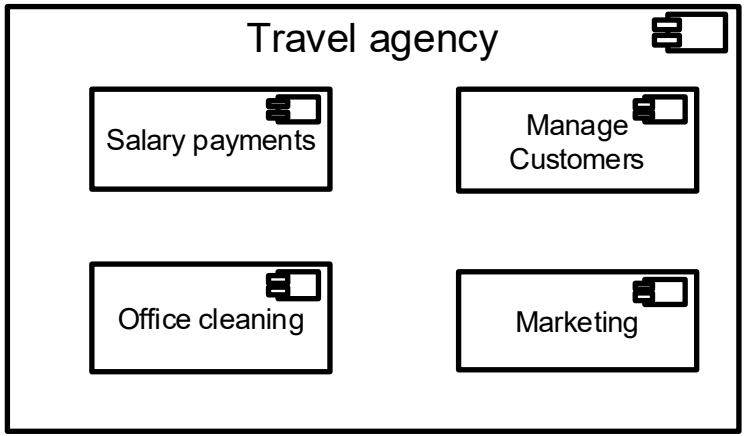
- High cohesion at Class level

Employee
Salary Email
SetSalary(newSalary) GetSalary() SetEmailAdress(newemail) GetEmailAddr()



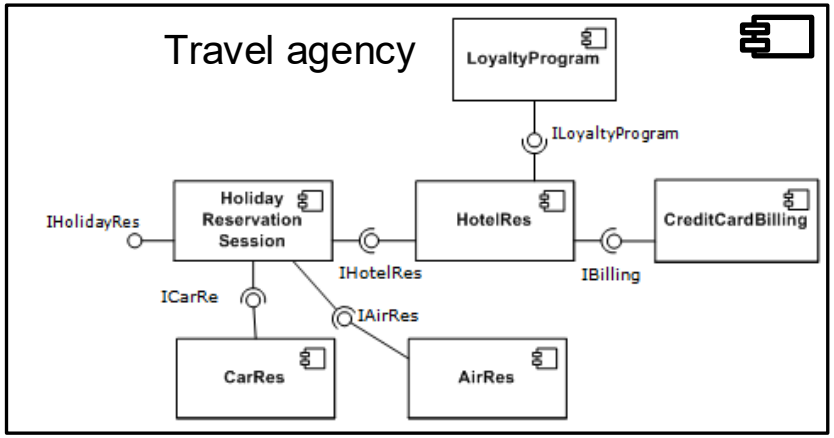
High Cohesion

- Low cohesion at Component level





- High cohesion at Component level





V. Low Coupling

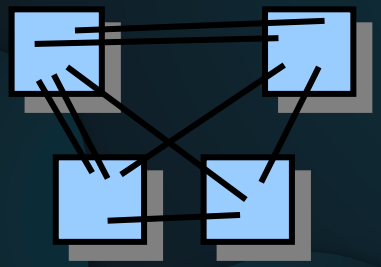
Aim for classes to have minimal dependencies on each other, facilitating easier maintenance and flexibility in the system.

…the degree of interdependence between components. The higher the coupling, the more components affect each other, and the more one component depends on changes in another.

A good example of following Low Coupling as responsibility of an interface to represent a contract of a component as a whole. All other components should rely solely on the interface and not depend on the component's internal implementation or structure.

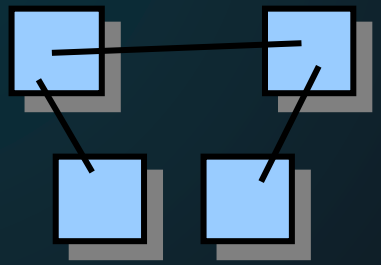
V. Low Coupling

Example: In the library management system, suppose there is a LibraryCatalog class responsible for managing the catalog of books. Instead of directly accessing the Book class to check availability, the LibraryCatalog class can rely on an interface, such as Searchable, implemented by the class Book. This way, LibraryCatalog class remains loosely coupled with `Book`, allowing for easier maintenance and changes.



high coupling





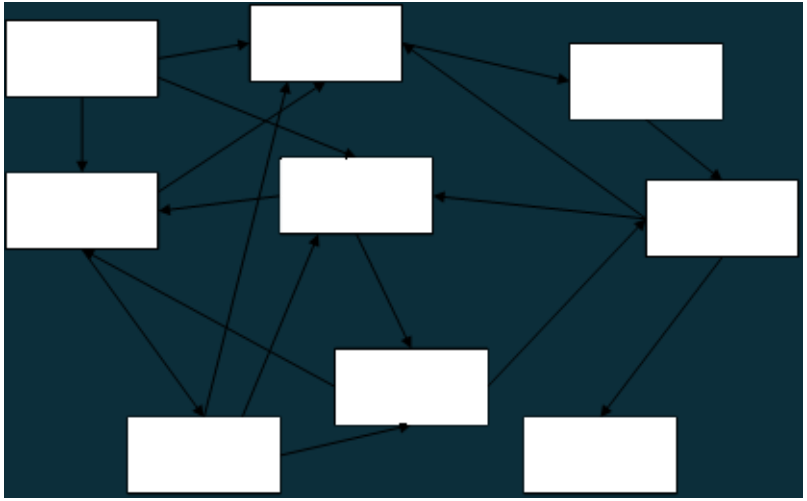
low coupling



Coupling

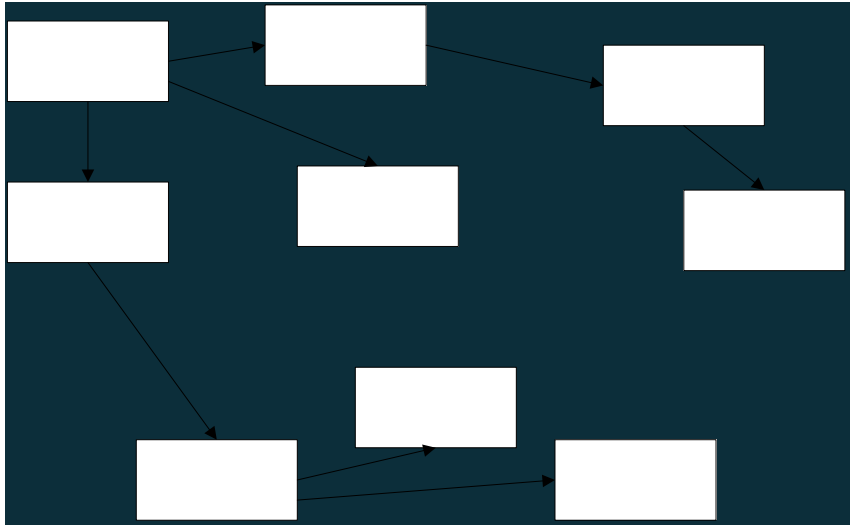
High coupling





- Low coupling





VI. Pure Fabrication

Introduce new classes to fulfill responsibilities without violating cohesion and coupling principles, promoting cleaner and more maintainable designs.

By isolating and encapsulating elements that don't naturally belong with the core business logic, Pure Fabrication helps maintain a clean separation between different parts of the system.

Example: Suppose the library management system needs to send email notifications to users when they borrow or return books. Instead of adding email sending logic directly to the Book or User classes, a separate NotificationService class can be created. This NotificationService class acts as a pure fabrication responsible for sending email notifications, maintaining low coupling and high cohesion in the system.

VII. Indirection

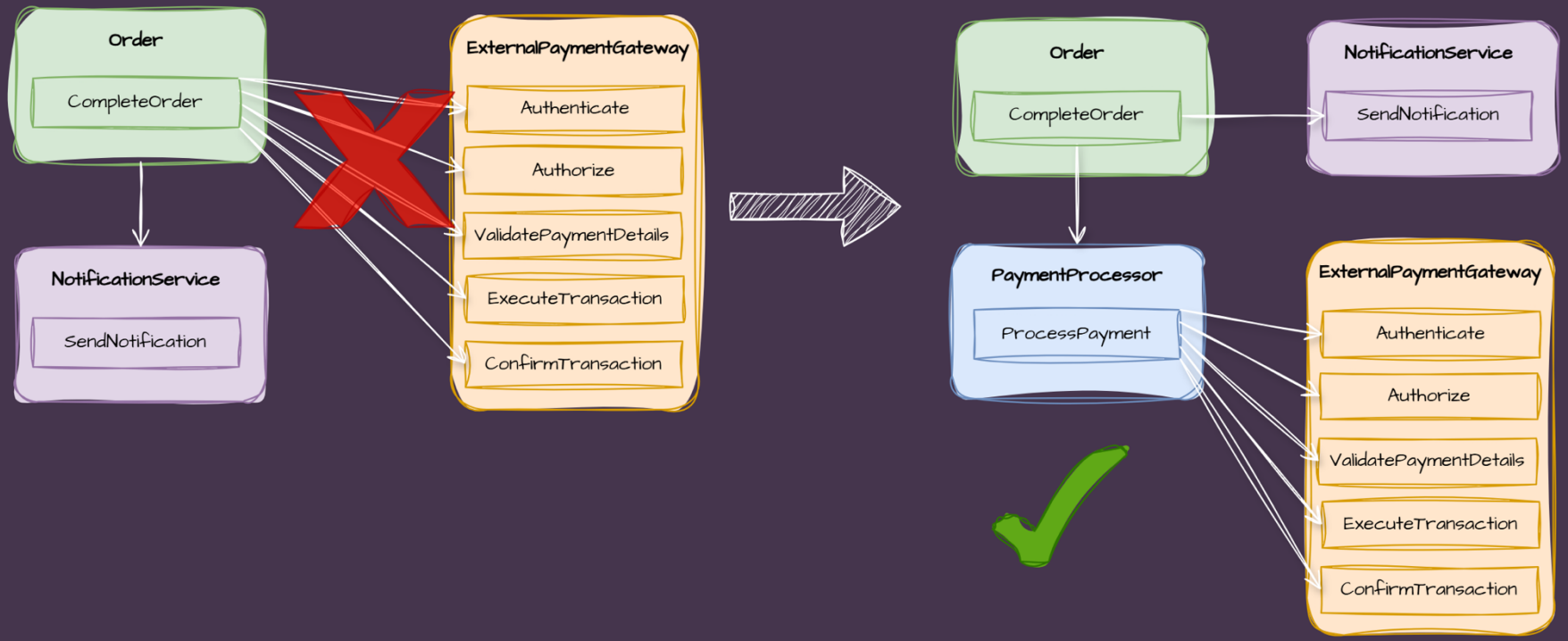
Use intermediaries or abstractions to decouple classes and promote flexibility in design. The mediator can be a component, an interface, an abstract class, or a design pattern.

Example: In the library management system, if multiple classes need to access book information, an BookRepository interface can be introduced. Classes that need access to book data can depend on the BookRepository interface rather than directly on the Book class. This allows for flexibility in how book information is retrieved, facilitating easier changes and adaptations in the future.

Indirection

bool.dev

Use an intermediary class to mediate between two classes to reduce coupling



Interactions between Order and ExternalPaymentGateway are complex, Which might cause an overload of logic inside the Order class

The Order class uses the PaymentProcessor to handle complex payment operations, such as authentication, authorization, and transaction execution, which reduces coupling by preventing Order from directly depending on the payment gateway and its intricate logic.

VIII. Polymorphism

Utilize inheritance and interfaces to enable multiple implementations of behaviors, allowing for flexible and extensible systems.

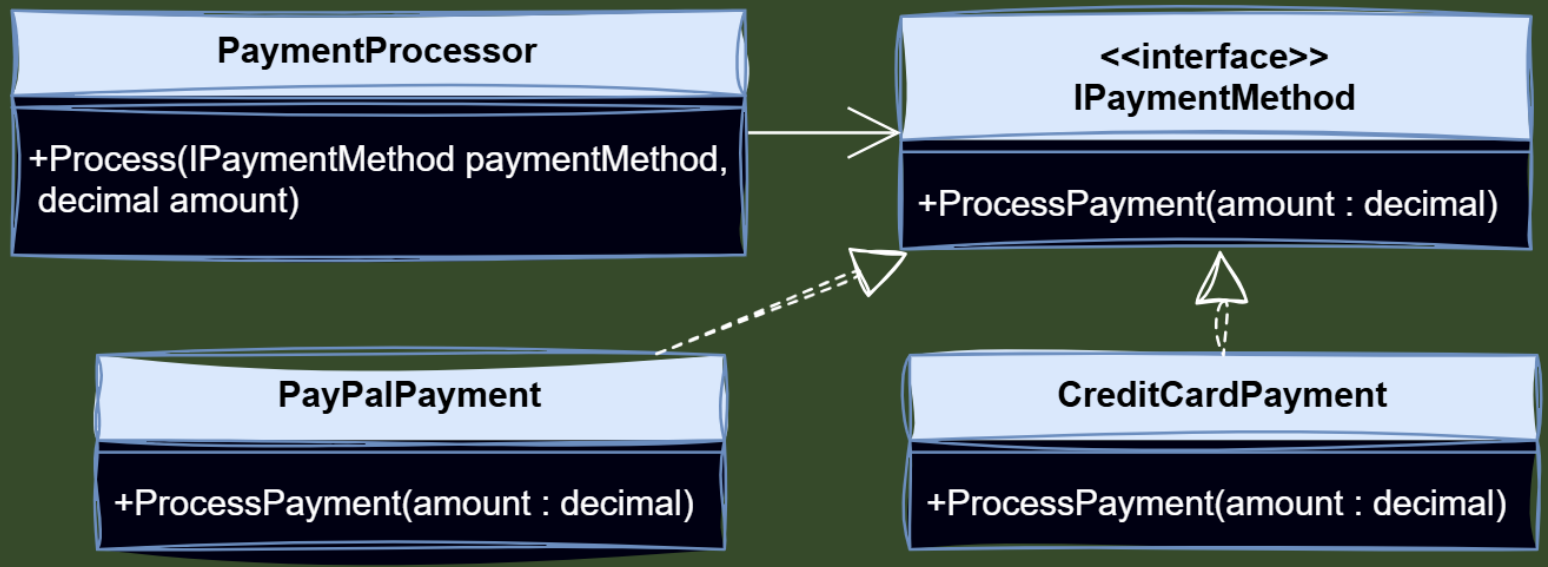
…to provide a way to perform a single action in different ways, allowing different objects to respond to the same message or method call.

Example: suppose there are different types of books, such as Fiction books and Non-Fiction books, each with its own borrowing rules. By defining a common interface, and implementing it in the FictionBook and NonFictionBook classes, polymorphism allows the borrowing process to be handled uniformly regardless of the book type. This promotes code reuse and simplifies the handling of different book types within the system.

Polymorphism

bool.dev

Use polymorphism to assign responsibilities for behaviors that may change depending on the type.

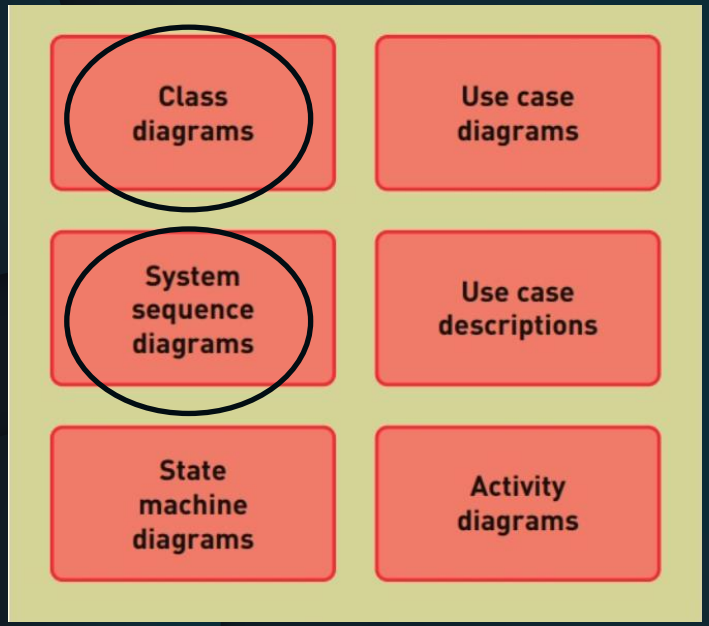


A payment system could use polymorphism for `CreditCardPayment`, `PayPalPayment`. These classes implement a common `IPaymentMethod` interface, each handling payments differently.

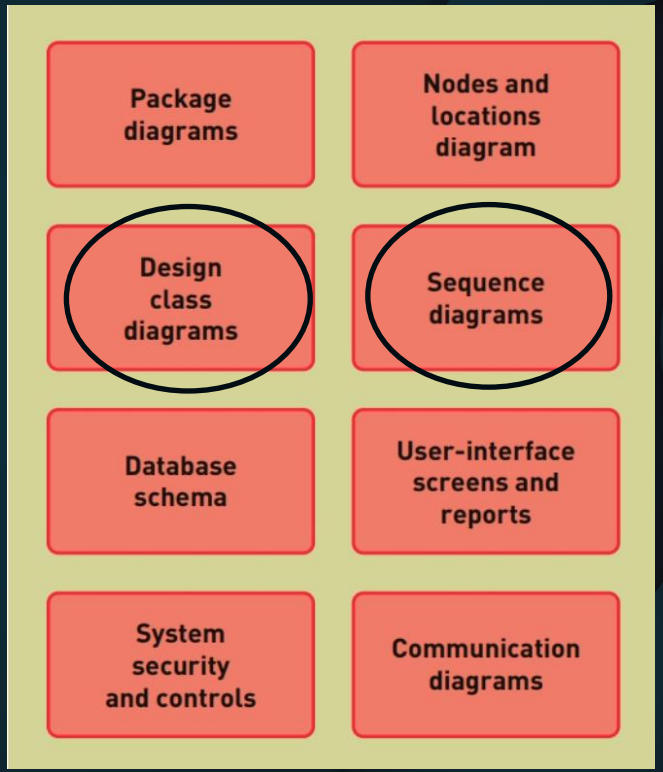
02

System sequence diagram*

analysis models



design models



In the object-oriented approach, the flow of information is achieved through sending messages either to and from actors or back and forth between internal objects.

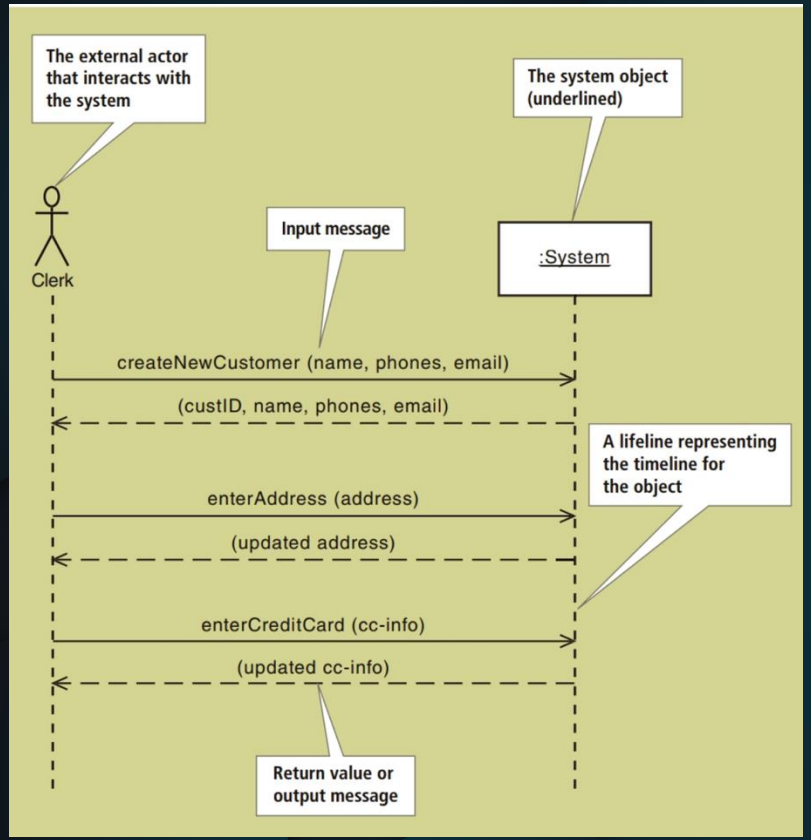
A **system sequence diagram (SSD)** is used to describe this flow of information into and out of the automated portion of the system.

It is an effective tool to help in the initial design of the user interface by identifying the specific information that *flows from the user into the system* and the information that *flows out of the system back to the user*.

Use case diagram: how the actor “uses” the system,

SSD: how the actor “interacts” with the system by entering input data and receiving output data.

System sequence diagram (SSD)



The most important information on an SSD is the sequence of messages between the actor and the system.

SSD is used to document the inputs to and outputs from the system for a single use case or scenario.

The system itself is treated as a single object named :System.

The inputs to the system are messages from the actor, and the outputs are usually return variables.

Loop frames, Alt frames, and Opt frames

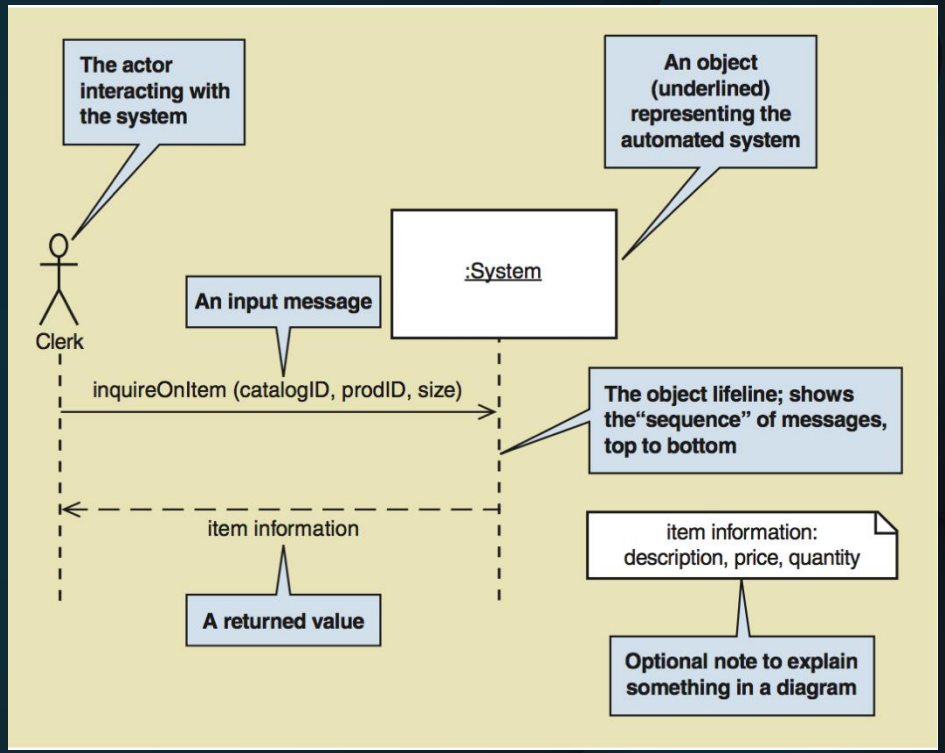
System sequence diagram

The box labeled :System is an object that represents the entire automated system.

A lifeline is the extension of that object—either actor or object. It denotes flow of time.

A message is labeled to describe its purpose and any input data being sent. (verb-noun syntax)

In this case, a valid response might be a list of all the information returned—for example, the description, price, and quantity of an item.



A note can be added to any UML diagram to add explanations.

When we have *repeating messages:*

a. detailed loop frame notation

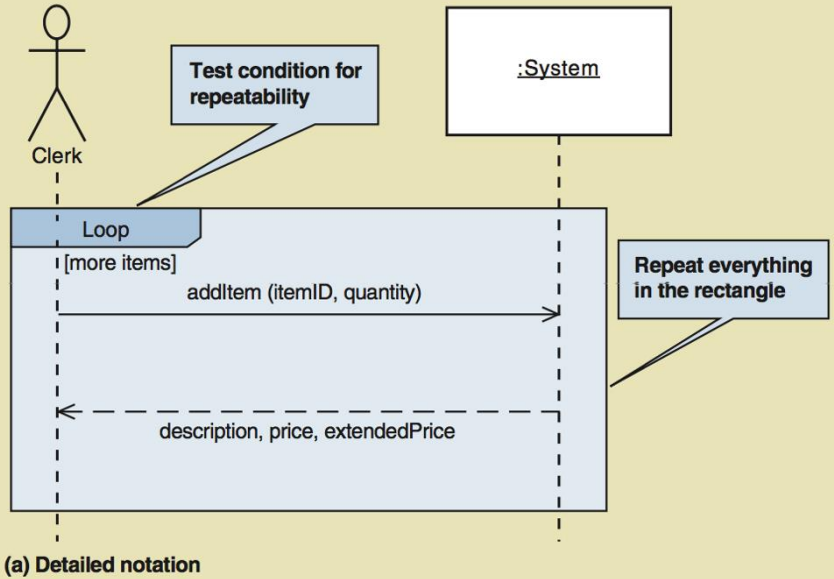
- indicates that the messages in the box repeat many times or are associated with many instances.

b. alternate notation

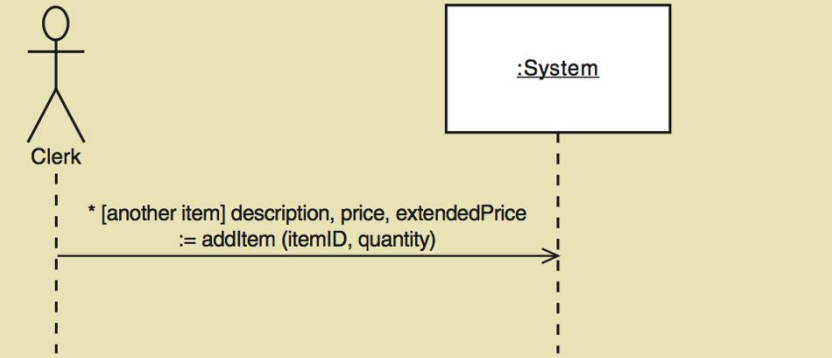
true/false condition - part of a message between objects that is *evaluated prior to transmission* to determine whether the message can be sent.

[true/false condition] return-value := message-name (parameter-list)

An asterisk (*) indicates repeating or looping of the message. Message-name is the description of the requested service written as a verb-noun.
Parameter-list shows the data that are passed with the message. Return-value describe data being returned from the destination object to the source object



(a) Detailed notation

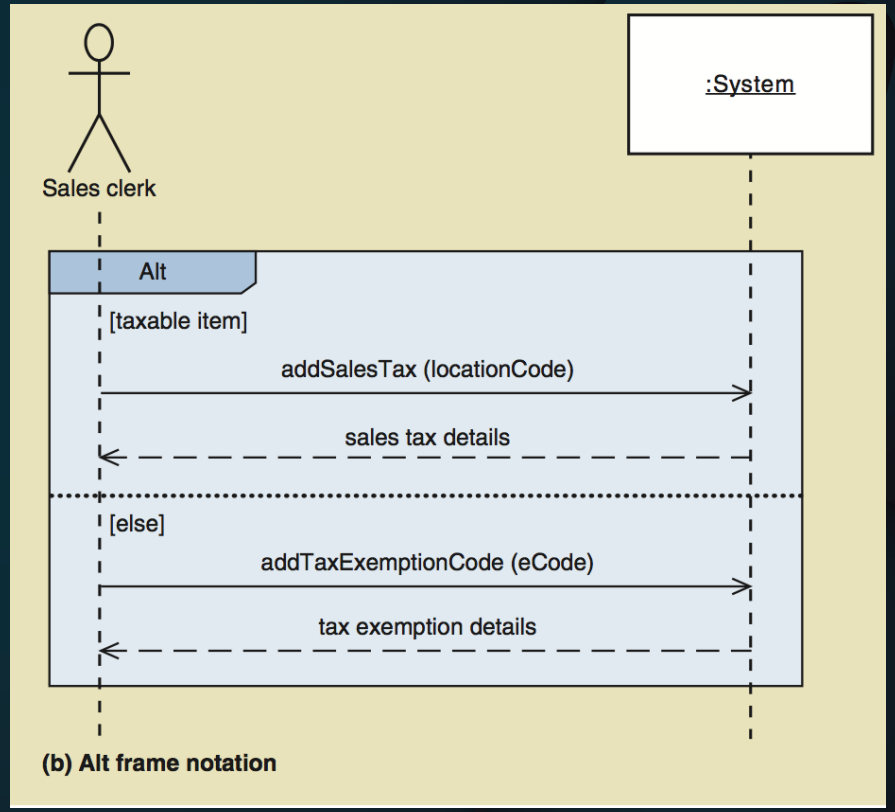


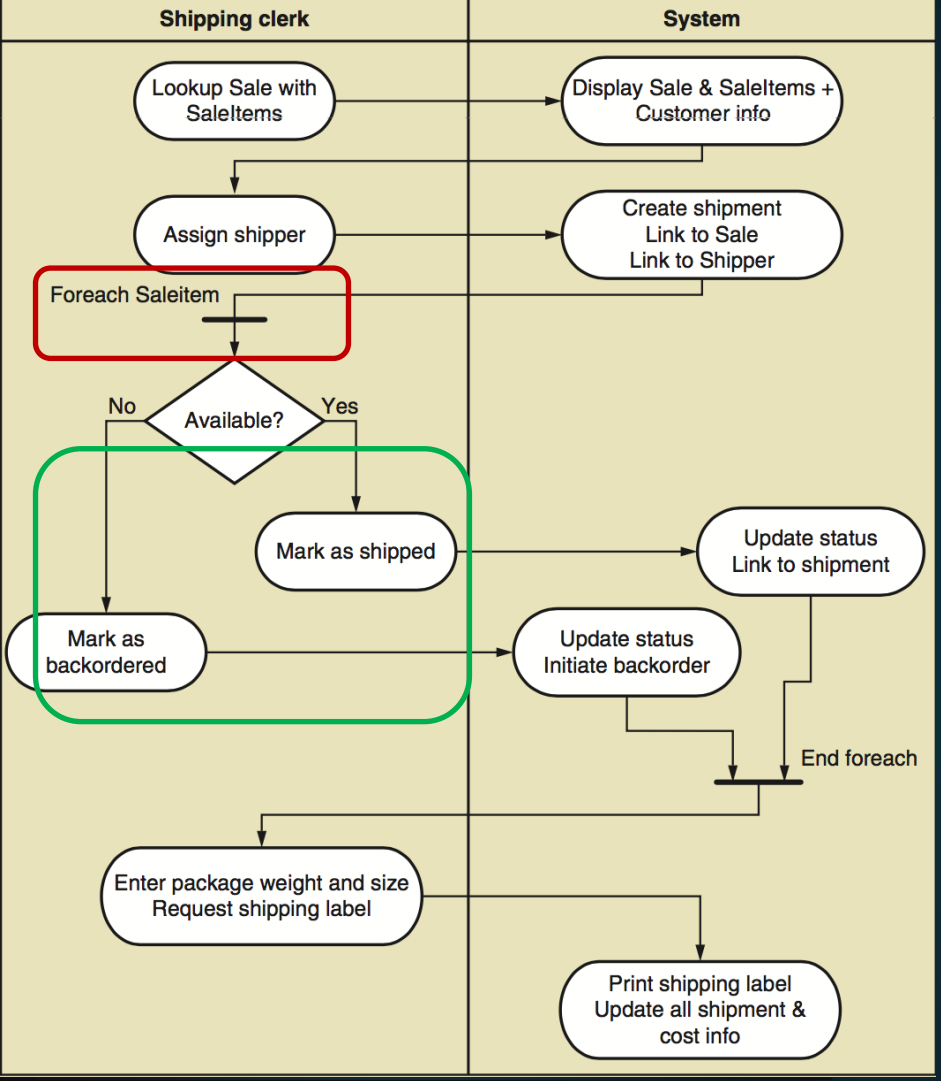
(b) Alternate notation

Alt frame

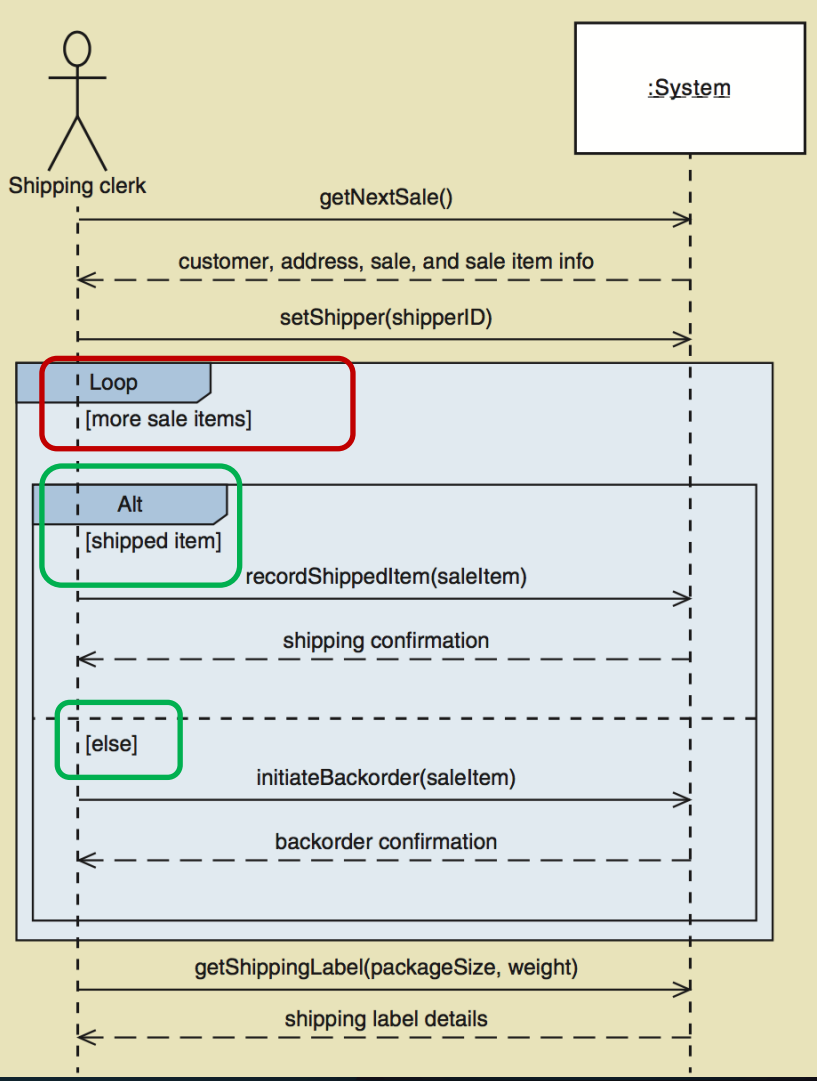
Alt frame notation on a sequence diagram showing if-then-else logic.

e.g., if an item is taxable, then add sales tax; otherwise, add a tax exemption code for a sales tax exemption.




Shipping clerk
System
Lookup Sale with SaleItems
Display Sale & SaleItems + Customer info
Assign shipper
Create shipment
Link to Sale
Link to Shipper
Foreach Saleitem
No
Available?
Yes
Mark as shipped
Update status
Link to shipment
Mark as backordered
Update status
Initiate backorder
End foreach
Enter package weight and size
Request shipping label
Print shipping label
Update all shipment & cost info

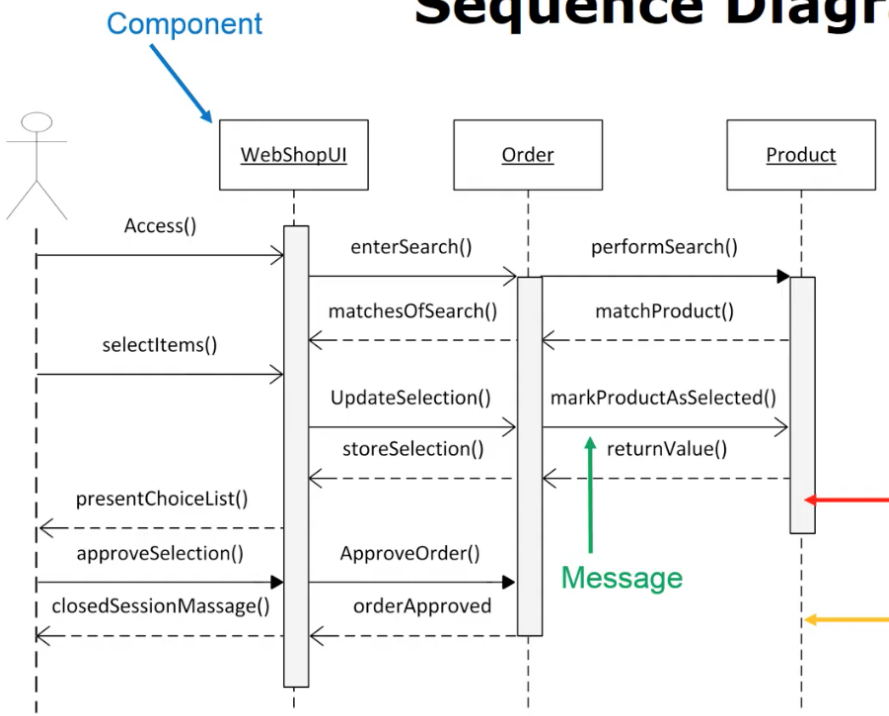
Example


:System
Shipping clerk
getNextSale()
customer, address, sale, and sale item info
setShipper(shipperID)
Loop
[more sale items]
Alt
[shipped item]
recordShippedItem(saleItem)
shipping confirmation
[else]
initiateBackorder(saleItem)
backorder confirmation
getShippingLabel(packageSize, weight)
shipping label details

03
Sequence diagram*

* will be required on the exam

Sequence Diagram



- Basic elements:
  - Actors/Components
  - Lifelines
  - Messages
  - Activation timelines

Activation timeline

Lifeline

first-cut Sequence diagram
- create customer account -

1. Expand the SSD – the :System object is replaced by all internal objects and messages within the system.

- to identify the internal objects that collaborate and the messages they send to each other, you need to carry out the use case (i.e., use case realization).

1.1. Look at the problem domain class diagram and create the first-cut design class diagram.

- extend the attributes with the type of information (data-type-expression) and make logical decisions about navigation visibility

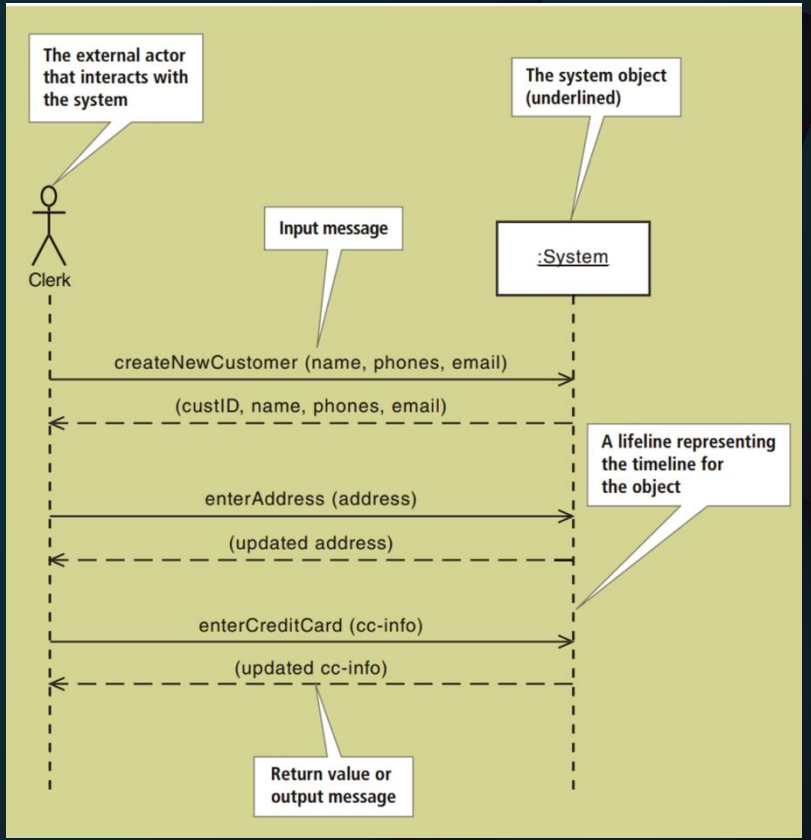
1.2. Draw a first-step sequence diagram with the identified problem domain objects and the input messages from the SSD

Use case name:	*Create customer account.*
Scenario:	Create online customer account.
Triggering event:	New customer wants to set up account online.
Brief description:	Online customer creates customer account by entering basic information and then following up with one or more addresses and a credit or debit card.
Actors:	Customer.
Related use cases:	Might be invoked by the *Check out shopping cart* use case.
Stakeholders:	Accounting, Marketing, Sales.
Preconditions:	Customer Account subsystem must be available. Credit/debit authorization services must be available.
Postconditions:	Customer must be created and saved. One or more Addresses must be created and saved. Credit/debit card information must be validated. Account must be created and saved. Address and Account must be associated with Customer.

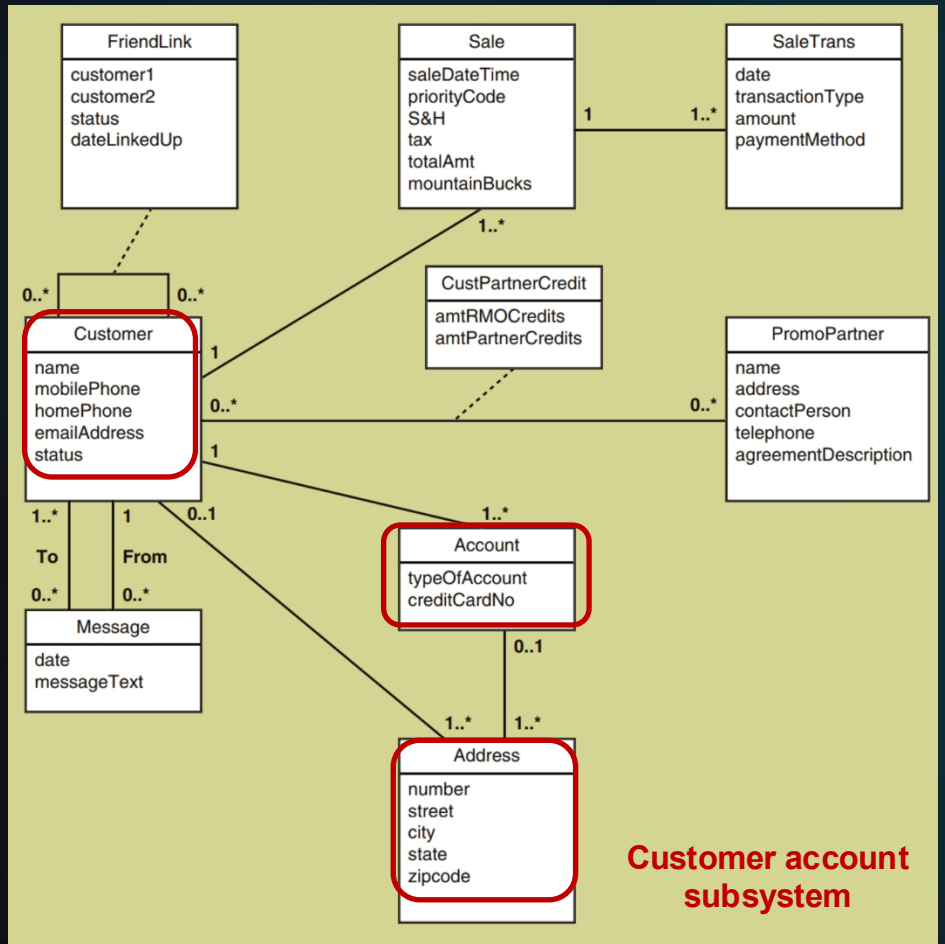
Flow of activities:	**Actor**	**System**
	1. Customer indicates desire to create customer account and enters basic customer information.	1.1 System creates a new customer. 1.2 System prompts for customer addresses.
	2. Customer enters one or more addresses.	2.1 System creates addresses. 2.2 System prompts for credit/debit card.
	3. Customer enters credit/debit card information.	3.1 System creates account. 3.2 System verifies authorization for credit/debit card. 3.3 System associates customer, address, and account. 3.4 System returns valid customer account details.

Exception conditions:	1.1 Basic customer data are incomplete. 2.1 The address isn't valid. 3.2 Credit/debit information isn't valid.

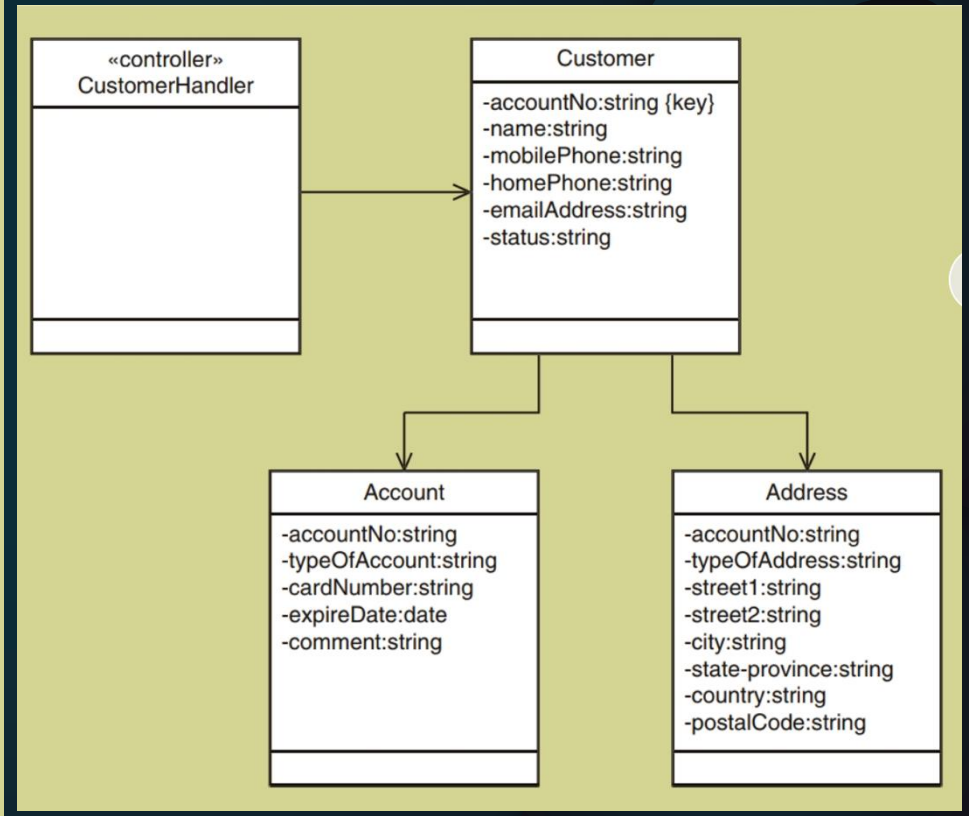
1. System sequence diagram



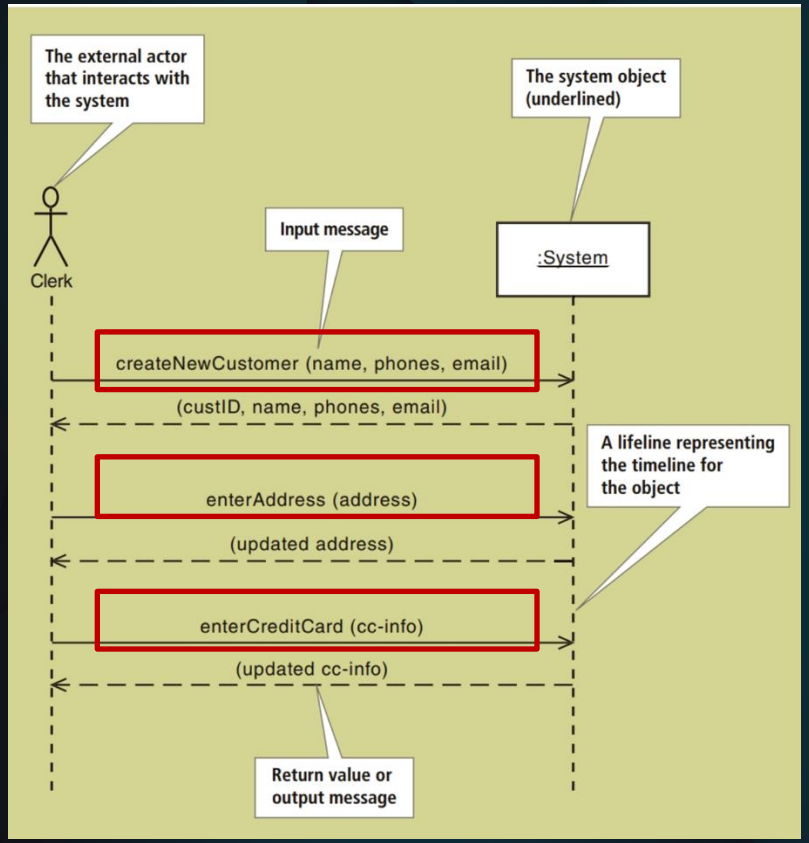
Domain class diagram



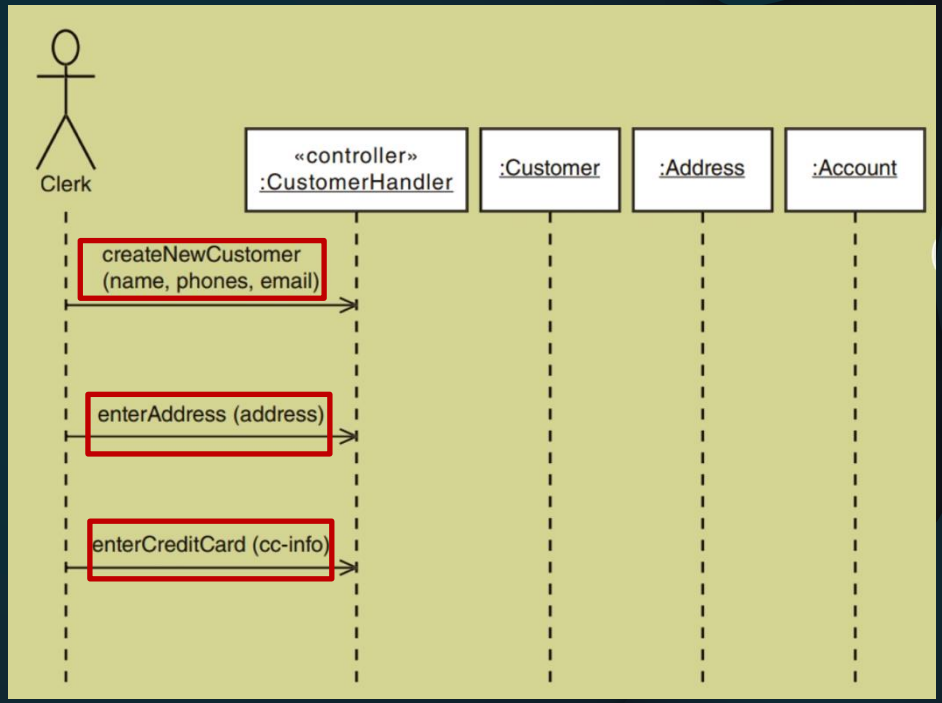
1.1. First-cut design class diagram



System sequence diagram



1.2. First-step in sequence diagram



first-cut Sequence diagram
- create customer account -

2. To determine the internal messages that must be sent between the objects, including which object should be the source and destination of each message.

2.1 follow the principles of good design: coupling, cohesion, object responsibility, etc.

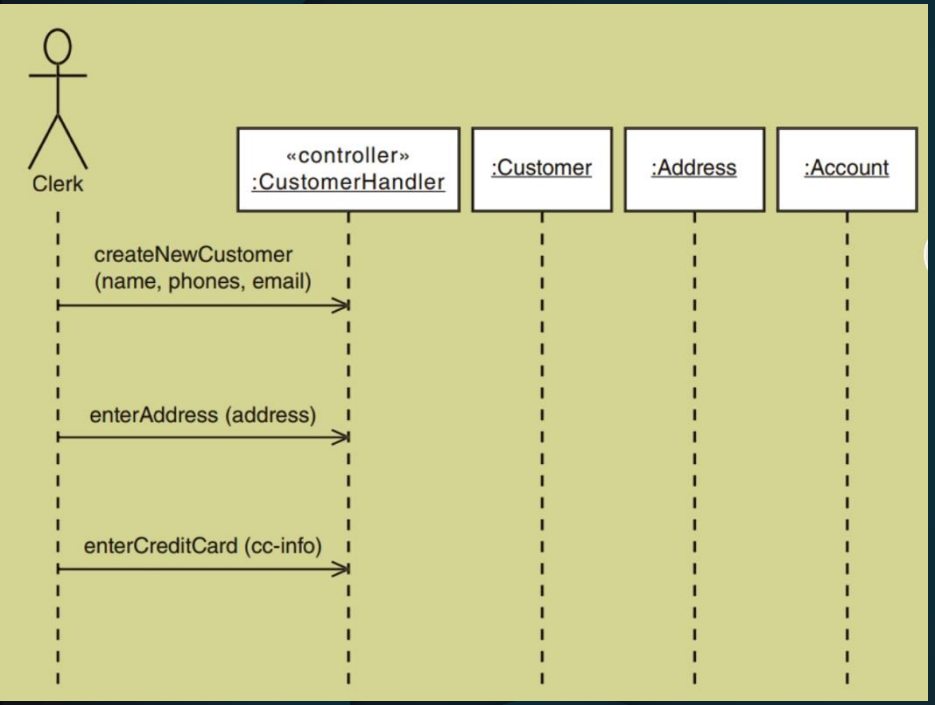
2.2 determine origin (carry out information) and destination (receives and process information) objects, and the passed parameters.

- because a message is requesting a service from the destination object, the message needs to reflect the requested service.

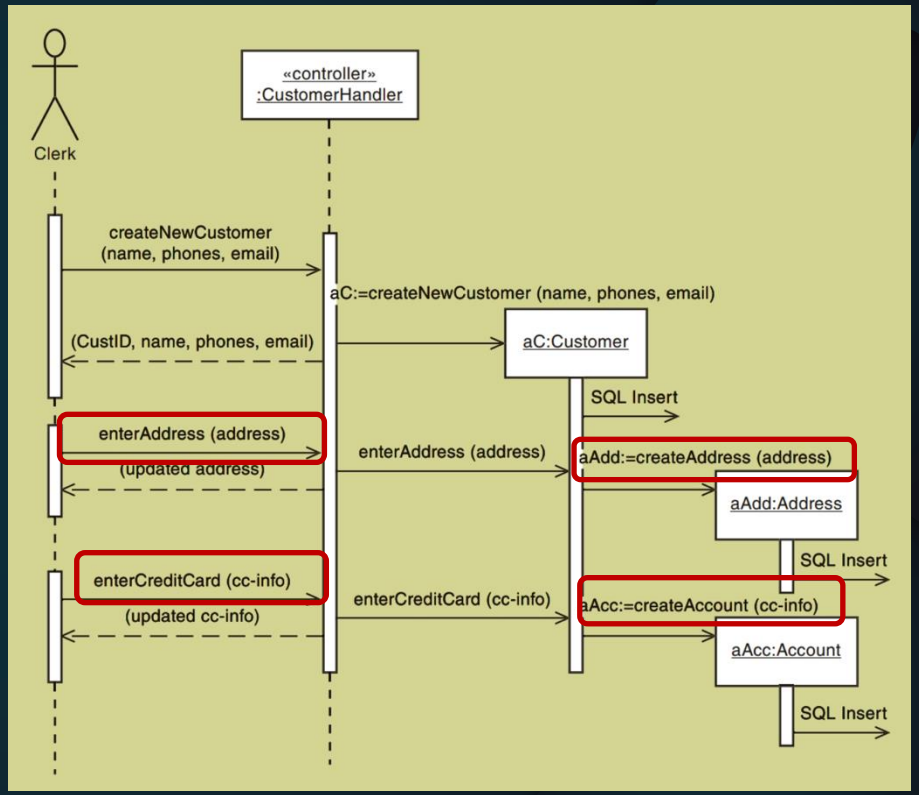
Note: first-cut sequence diagram focuses only on the domain classes.

It is important for the first-cut sequence diagram to identify all the domain classes and the required internal messages between them.

1.2. First-step in sequence diagram

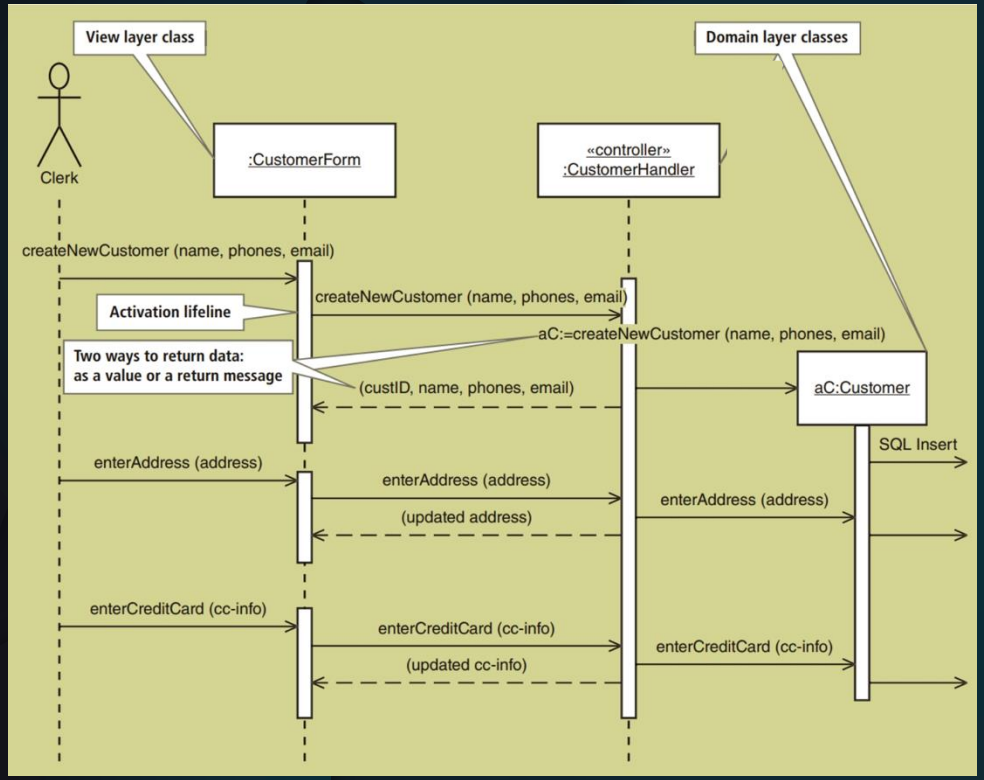


2.2. First-cut sequence diagram



return-value := message-name (parameter-list)

Sequence diagram (partial)



There are two ways to indicate data being returned, either by an object reference returned with the ":=" operator, or as a return message using a dashed arrow.

The primary objective of the sequence diagram is to identify which classes collaborate and what messages they must send to each other.

two-level detailed design for the use case: create customer account.

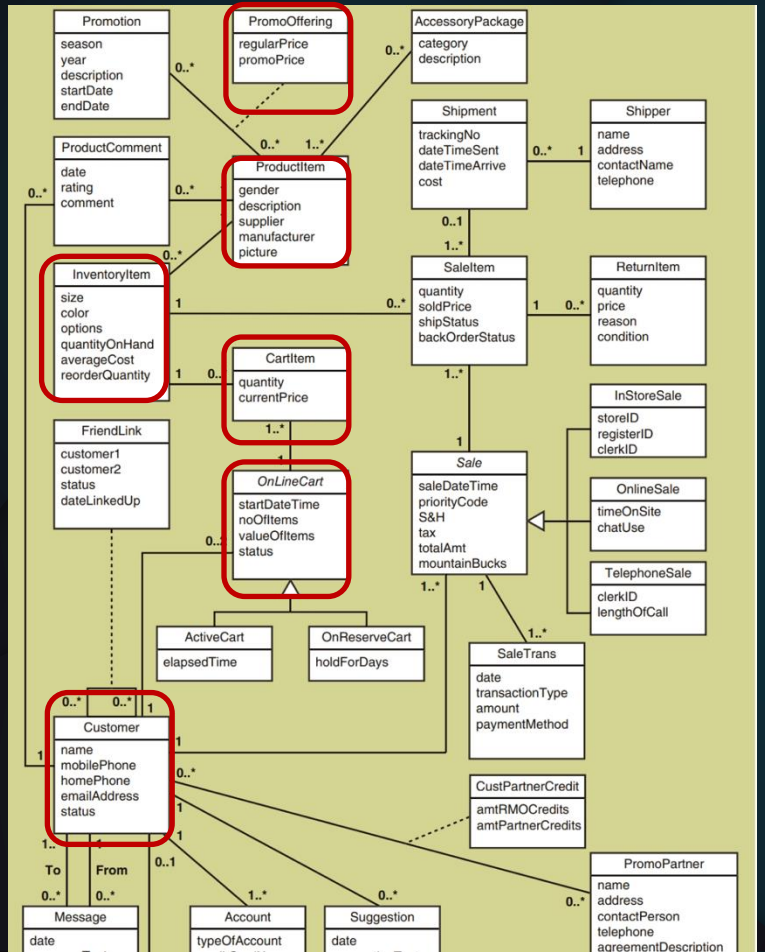
Note: the input messages from the external actor always go to the view layer objects.

label aC:Customer - the box represents a customer object with a reference variable name of aC.

I. aC := createNewCustomer (name, phone, email)
II. (custID, name, phone, email)

activation lifeline is a representation of the period during which a method of an object is alive and executing

1.1. Look at the domain class diagram and create the first-cut design class diagram.



- Use case: *fill shopping cart*

- The Customer, Cart, and CartItem

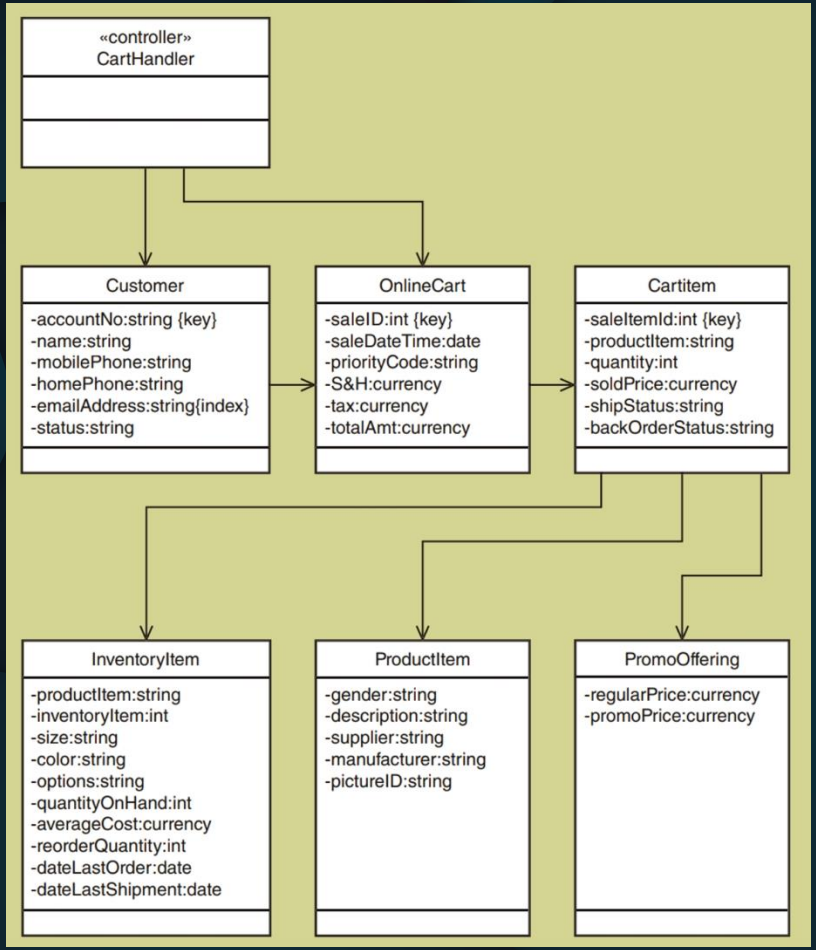
the use case will be adding items for the customer to the customer's cart.

To create a cart item the system needs to know:

what product the item belong to, if there are items in stock, and the price for the items.

- The ProductItem, InventoryItem, and PromoOffering

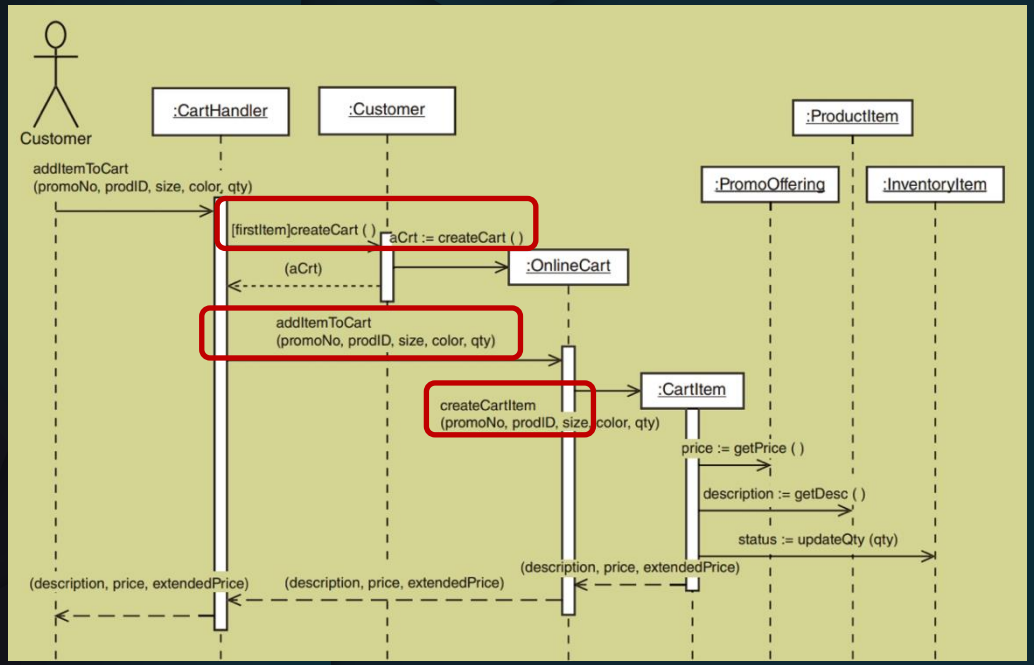
How many classes are going to be in the first-cut design class diagram?



first-cut design class diagram

Use case: add item to a cart

first-cut Sequence diagram



1. The controller object determines if this is the very first item for this customer and, if so, sends a message to :Customer to create a new online cart.

Note: In UML, when create message is sent to an object, it is often drawn directly to the object's box and not to the lifeline.

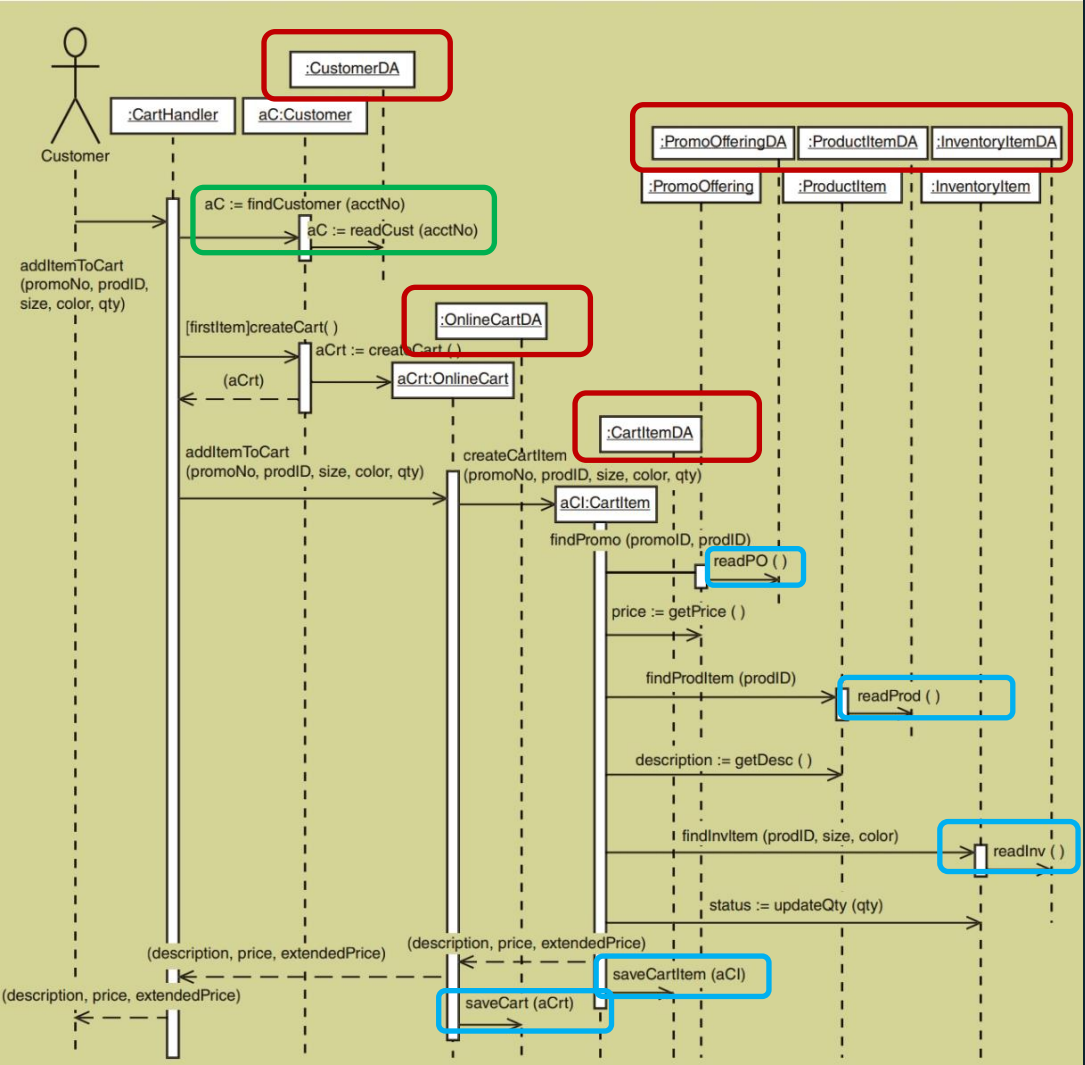
2. The controller sends addItemToCart message to the :OnlineCart.

3. The :OnlineCart processes that message by creating a :CartItem.

4. The :CartItem then takes responsibility to get its price and description and see that the inventory is updated.

[true/false condition] return-value := message-name (parameter-list)

Design principle: objects that "own" other objects are responsible for creating/initilizing those objects.



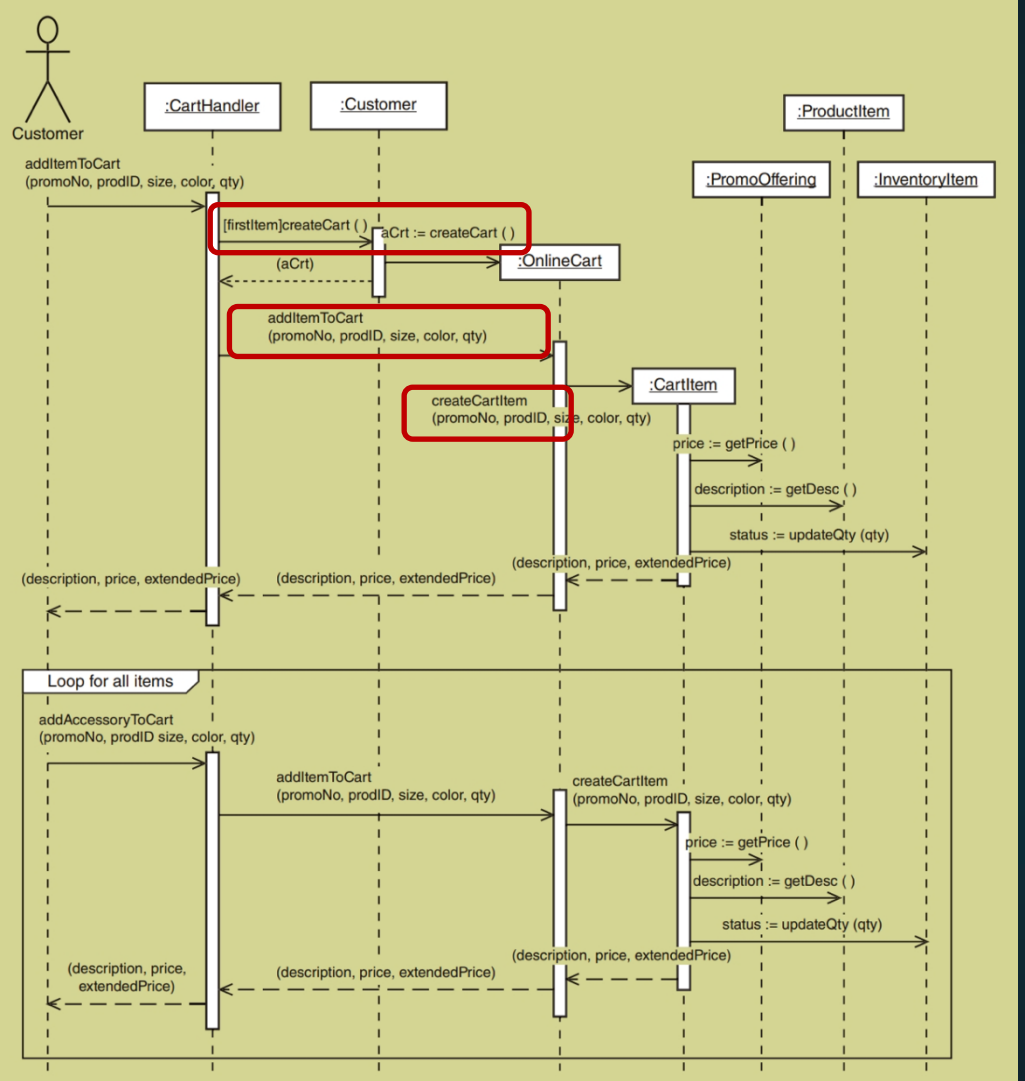
What is different – each problem domain object is associated with a data access object; and each original internal message is associated with additional messages to retrieve data from or save data to the database via the data access object.

Note: every object that sends a message to another object must have navigation visibility to that object.

getPrice, getDesc, updateQty – require previous message to find the appropriate data from the DB.

The primary thing to remember is that data access objects are necessary to retrieve data.

[true/false condition] return-value := message-name (parameter-list)



first-cut Sequence diagram

1. The controller object determines if this is the very first item for this customer and, if so, sends a message to :Customer to create a new online cart.

Note: In UML, when create message is sent to an object, it is often drawn directly to the object's box and not to the lifeline.

2. The controller sends addItemToCart message to the :OnlineCart.

3. The :OnlineCart processes that message by creating a :CartItem.

4. The :CartItem then takes responsibility to get its price and description and see that the inventory is updated.

Design principle: objects that "own" other objects are responsible for creating those objects.

Does anyone have any questions?

THANK YOU!

katerina.mangaroska@usn.no